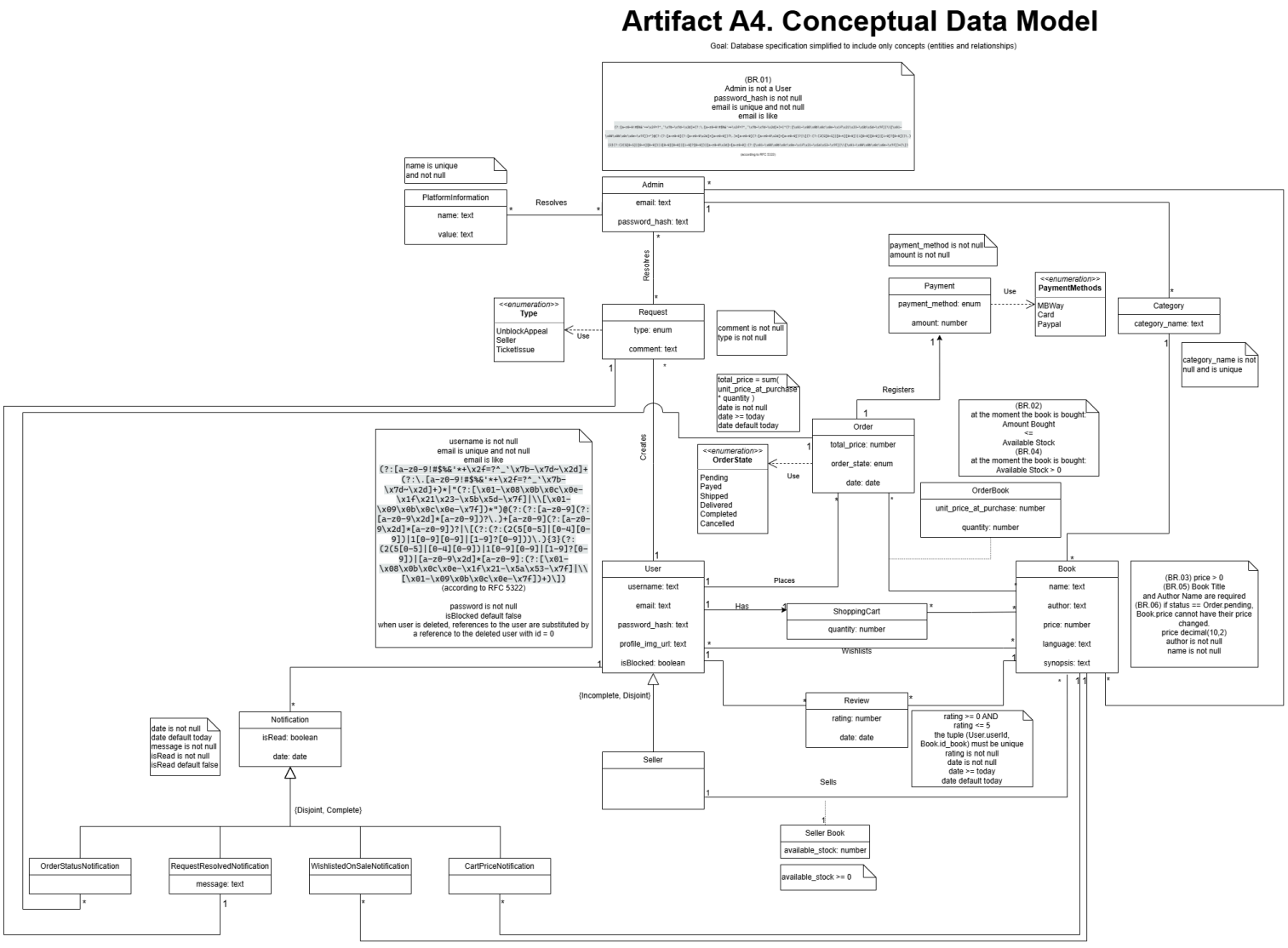
EBD: Database Specification Component

> Designed to redefine the reading experience, Liberato brings together technology, accessibility, and community to create a modern space for book lovers. The platform allows readers to discover, purchase, and review books with ease. By giving costumers a seamless shopping experience, Liberato empowers readers, authors, and publishers to connect through a shared passion for literature.

A4: Conceptual Data Model

> Designed to enhance the book community experience, Liberato allows readers to discover, purchase, and review books with ease. The following sections present a detailed overview of the project's conceptual data model, including class diagrams and business rules.

1. Class diagram



2. Additional Business Rules

Identifier	Name	Description
BR01	Administrator Role Separation	Administrator accounts are independent of user accounts and cannot buy products (i.e., they have no purchasing capability).
BR02	Request of books Pre-Requisite	At the moment a book is bought, the amount of books requested to be purchased has to be lower than the amount of books in stock.
BR03	Inventory Pre-Requisite	Buyers can only complete a purchase for books that are currently in stock (Stock > 0).
BR04	Book Price Validation	The price of a book must be greater than zero to ensure all listed books have a valid and positive value.
BR05	Book Identification Requirement	Each book must have a title and an author to be registered in the system, ensuring all entries are properly identified and traceable.

A5: Relational Schema, validation and schema refinement

> The goal of this artifact is to define the logical structure of the system's database and ensure its consistency, normalization, and alignment with the data requirements identified in previous stages. This artifact presents the relational schema, describing entities, attributes, and relationships, followed by validation to confirm that the schema accurately represents the conceptual model. Finally, schema refinement is performed to eliminate redundancies, improve integrity, and optimize data organization, ensuring a robust and efficient database foundation for system implementation.

1. Relational Schema

> The Relational Schema includes the relation schemas, attributes, domains, primary keys, foreign keys and other integrity rules: UNIQUE, DEFAULT, NOT NULL, CHECK.
> Relation schemas are specified in the compact notation:

Relation reference	Relation Compact Notation
R01	user(id, email **UN NN CK** email **LIKE** Email, username **NN**, password_hash **NN**, profile_img_url, isBlocked **NN DF** false)
R02	seller(user_id -> user **NN UK**)
R03	notification(id, user_id -> user **NN**, message **NN**, date **NN DF** Today, is_read **NN DF** false)
R04	book(id, category_id -> category **NN**, name **NN**, author **NN**, price **NN CK** price > 0, language **NN**, synopsis)
R05	seller_book(seller_id -> seller, book_id -> book, available_stock **NN CK** available_stock >= 0)
R06	review(user_id -> user **NN**, book_id -> book **NN**, rating **NN CK** 0 <= rating AND rating <= 5, date **NN DF** Today **CK** date >= Today)
R07	order(id, user_id -> user **NN**, payment_id -> payment **NN UK**, date **NN DF** Today **CK** date >= Today, total_price **NN CK** total_price > 0 AND total_price = sum(order_book.unit_price_at_purchase*order_book.quantity))
R08	order_book(order_id -> order, book_id -> book, unit_price_at_purchase **NN CK** unit_price_at_purchase > 0, quantity **NN CK** quantity > 0)
R09	payment(order_id -> order, amount **NN CK** amount > 0, payment_method **NN CK** payment_method **IN** PaymentMethods)
R10	category(id, name **NN UK**)
R11	admin(id, email **NN UK CK** email **LIKE** Email, password_hash **NN**)
R12	request(id, user_id -> user **NN**, admin_id -> admin **NN**, type **NN DF** TicketIssue **CK** type **IN** RequestTypes, comment **NN**, status **NN DF** pending **CK** status **IN** Status)
R13	platform_information(id, name **NN UK**, value)
R14	shopping_cart(id, user_id -> user **NN**, book_id -> book **NN**, quantity **NN CK** quantity > 0)
R15	wishlist(user_id -> user, book_id -> book

2. Domains

> The specification of additional domains can also be made in a compact form, using the notation:

Domain Name	Domain Specification
Today	DATE DEFAULT CURRENT_DATE
RequestTypes	ENUM ("unblock_appeal", "seller", "ticket_issue")
PaymentMethods	ENUM ("MBWay", "Card", "Paypal")
Status	ENUM ("pending", "resolved")

Domain Name	Domain Specification
Email	VARCHAR(255) CK(VALUE LIKE '(?:[a-z0-9!#%&'*+\\x2f=?^_\`\\x7b-\\x7d\~\\x2d\]+(?:\\.\[a-z0-9!#%&'*+\x2f=?^_`\x7b-\x7d~\x2d]+)*|"(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\[\x01-\x09\x0b\x0c\x0e-\x7f])*")@(?:(?:a-z0-9?\.)+a-z0-9?|\[(?:(?:(2(5[0-5]|[0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9]))\.){3}(?:(2(5[0-5]|[0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9])|[a-z0-9\x2d]*[a-z0-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])+)\])

3. Schema validation

> To validate the Relational Schema obtained from the Conceptual Model, all functional dependencies are identified and the normalization of all relation schemas is accomplished. Should it be necessary, in case the scheme is not in the Boyce–Codd Normal Form (BCNF), the relational schema is refined using normalization.

TABLE R01	User
Keys	{id}, {email}
Functional Dependencies	
FD0101	id → {email, username, password_hash, profile_img_url, isBlocked}
FD0102	email → {id, username, password_hash, profile_img_url, isBlocked}
NORMAL FORM	BCNF

> Each determinant is a candidate key; all non-key attributes depend only on keys.

TABLE R02	Seller
Keys	{user_id}
Functional Dependencies	
FD0201	user_id → {}
NORMAL FORM	BCNF

TABLE R03	Notification
Keys	{id}
Functional Dependencies	
FD0301	id → {user_id, message, date, is_read}
NORMAL FORM	BCNF

TABLE R04	Book
Keys	{id}
Functional Dependencies	
FD0401	id → {category_id, name, author, price, language, synopsis}
NORMAL FORM	BCNF

TABLE R05	Seller_Book
Keys	{seller_id, book_id}
Functional Dependencies	

TABLE R05	Seller_Book
FD0501	{seller_id, book_id} → {available_stock}
NORMAL FORM	BCNF

TABLE R06	Review
Keys	{user_id, book_id}
Functional Dependencies	
FD0601	{user_id, book_id} → {rating, date}
NORMAL FORM	BCNF

TABLE R07	Order
Keys	{id}
Functional Dependencies	
FD0701	id → {user_id, payment_id, date, total_price}
FD0702	payment_id → id
NORMAL FORM	BCNF

TABLE R08	Order_Book
Keys	{order_id, book_id}
Functional Dependencies	
FD0801	{order_id, book_id} → {unit_price_at_purchase, quantity}
NORMAL FORM	BCNF

TABLE R09	Payment
Keys	{order_id}
Functional Dependencies	
FD0901	order_id → {amount, payment_method}
NORMAL FORM	BCNF

TABLE R10	Category
Keys	{id}, {name}
Functional Dependencies	
FD1001	id → {name}
FD1002	name → {id}
NORMAL FORM	BCNF

TABLE R11	Admin
Keys	{id}, {email}
Functional Dependencies	
FD1101	id → {email, password_hash}
FD1102	email → {id, password_hash}
NORMAL FORM	BCNF

TABLE R12	Request
Keys	{id}
Functional Dependencies	
FD1201	id → {user_id, admin_id, type, comment, status}
NORMAL FORM	BCNF

TABLE R13	Platform Information
Keys	{id}, {name}
Functional Dependencies	
FD1301	id → {name, value}
FD1302	name → {id, value}
NORMAL FORM	BCNF

TABLE R14	Shopping Cart
Keys	{id}, {user_id, book_id}
Functional Dependencies	
FD1401	id → {user_id, book_id, quantity}
FD1402	{user_id, book_id} → {id, quantity}
NORMAL FORM	BCNF

TABLE R15	Wishlist
Keys	{user_id, book_id}
Functional Dependencies	
FD1501	{user_id, book_id} → {}
NORMAL FORM	BCNF

A6: Indexes, triggers, transactions and database population

> The goal of this artifact is to enhance the functionality, integrity, and performance of the database through the implementation of essential database mechanisms. It defines and justifies the use of indexes to optimise query performance, triggers to automate data validation and consistency tasks, and transactions to ensure reliable and atomic operations. Additionally, the database population phase demonstrates the initial data setup, providing realistic datasets for testing, validation, and future development stages. Together, these components strengthen the database's efficiency, reliability, and readiness for deployment.

1. Database Workload

> A study of the predicted system load (database load). Estimate of tuples at each relation.

Relation reference	Relation Name	Order of magnitude	Estimated growth
R01	user	tens of thousands	dozens per day
R02	seller	tens	units per week
R03	notification	tens of thousands	dozens per day
R04	book	hundreds	dozens per week
R05	seller_book	hundreds	dozens per week
R06	review	tens of thousands	dozens per day
R07	order	tens of thousands	dozens
R08	order_book	tens of thousands	dozens
R09	payment	tens of thousands	dozens
R10	category	units	no growth
R11	admin	units	no growth
R12	request	units	units per week
R13	platform_information	units	no growth
R14	shopping_cart	hundreds	units per day
R15	wishlist	tens of thousands	dozens per day

2. Proposed Indices

2.1. Performance Indices

> Indices proposed to improve performance of the identified queries.

Index	Relation	Attribute	Type	Cardinality	Clustering	Justification
IDX01	`notification`	`user_id`	B-tree	High	No	Speeds up queries that list notifications from a specific user.
IDX02	`request`	`user_id`	B-tree	Medium	No	Improves the performance of queries that list requests made by a user.
IDX03	`request`	`resolved_by`	B-tree	Innovation	No	It facilitates the listing of requests resolved by an administrator.

Index	Relation	Attribute	Type	Cardinality	Clustering	Justification
IDX04	`book`	`category_name`	B-tree	High	No	It optimizes the listing of books by category, a common practice in searches by literary genre.
IDX05	`book`	`seller_id`	B-tree	Medium	No	It improves the performance of queries that list books sold by a specific seller.
IDX06	`review`	`book_id`	B-tree	High	No	Speeds up the loading of reviews associated with each book.
IDX07	`wishlist`	`user_id`	B-tree	Medium	No	Speeds up queries to list a user's wishlist.
IDX08	`shopping_cart`	`user_id`	B-tree	Medium	No	Improves performance when listing items in a user's shopping cart.
IDX09	`orders`	`user_id`	B-tree	High	No	Speeds up the listing and viewing of a user's orders.
IDX10	`order_book`	`book_id`	B-tree	High	No	It facilitates inquiries about sales of a specific book.
IDX11	`seller_book`	`seller_id`	B-tree	Medium	No	Optimizes access to books belonging to a specific seller.

2.2. Full-text Search Indices

> The system being developed must provide full-text search features supported by PostgreSQL. Thus, it is necessary to specify the fields where full-text search will be available and the associated setup, namely all necessary configurations, indexes definitions and other relevant details.

Index	IDX01
Relation	book
Attribute	name

Index	IDX01
Type	GiST
Clustering	No
Justification	it improves overall performance of full-text searches of books by name; GiST is better for substring matching
`SQL code`	`CREATE INDEX book_search ON book USING GIST (name);`

3. Triggers

> User-defined functions and trigger procedures that add control structures to the SQL language or perform complex computations, are identified and described to be trusted by the database server. Every kind of function (SQL functions, Stored procedures, Trigger procedures) can take base types, composite types, or combinations of these as arguments (parameters). In addition, every kind of function can return a base type or a composite type. Functions can also be defined to return sets of base or composite values.

Trigger	TRIGGER01
Description	This trigger implements the business rule that indicates that the available stock of a book should be updated when an order is placed. Whenever a new record is inserted into order_book, the corresponding stock in book is decreased according to the quantity purchased, ensuring that the inventory reflects the actual number of books available.
`SQL code`	

```
CREATE OR REPLACE FUNCTION update_book_stock_after_order()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE book
    SET available_stock = available_stock - NEW.quantity
    WHERE book_id = NEW.book_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_book_stock
AFTER INSERT ON order_book
FOR EACH ROW
EXECUTE FUNCTION update_book_stock_after_order();`
```

|

Trigger	TRIGGER02
Description	This trigger prevents a book's stock from becoming negative. Before inserting a new line into order_book, it checks if the requested quantity exceeds the existing stock. If it does, the operation is canceled and an exception is thrown. This ensures data integrity and prevents the sale of out-of-stock books.
`SQL code`	

```
CREATE OR REPLACE FUNCTION prevent_negative_stock()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT available_stock FROM book WHERE book_id = NEW.book_id) - NEW.quantity < 0
        RAISE EXCEPTION 'Stock insuficiente para o livro %', NEW.book_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_negative_stock
BEFORE INSERT ON order_book
```

```
FOR EACH ROW
EXECUTE FUNCTION prevent_negative_stock();
```

|

Trigger	TRIGGER03
Description	This trigger keeps the total price of an order updated. Whenever a new book is added to the order_book table, the total value in the orders table is automatically incremented according to the unit price and quantity purchased. This prevents inconsistencies between the order items and the recorded total value.
`SQL code`	

```
CREATE OR REPLACE FUNCTION update_order_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE orders
    SET total_price = total_price + (NEW.unit_price_at_purchase * NEW.quantity)
    WHERE order_id = NEW.order_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_order_total
AFTER INSERT ON order_book
FOR EACH ROW
EXECUTE FUNCTION update_order_total();
```

|

Trigger	TRIGGER04
Description	This trigger applies the business rule associated with account unlock requests (UnblockAppeal). When a new request of this type is created in the request table, the corresponding user is automatically marked as blocked (is_blocked = TRUE), ensuring consistency between the account status and the pending request.
`SQL code`	

```
CREATE OR REPLACE FUNCTION auto_block_user()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.type = 'UnblockAppeal' THEN
        UPDATE users
        SET is_blocked = TRUE
        WHERE user_id = NEW.user_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_block_user
AFTER INSERT ON request
FOR EACH ROW
EXECUTE FUNCTION auto_block_user();
```

|

Trigger	TRIGGER05
Description	This trigger synchronises the available stock between the book table and the seller_book table. Whenever a book's stock changes in the book table, the change is propagated to the corresponding records in seller_book. This ensures consistency between the overall stock and the individual stock of each seller.

Trigger	TRIGGER05
`SQL code`	

```
CREATE OR REPLACE FUNCTION sync_seller_stock()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE seller_book
    SET available_stock = NEW.available_stock
    WHERE book_id = NEW.book_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_seller_stock
AFTER UPDATE OF available_stock ON book
FOR EACH ROW
EXECUTE FUNCTION sync_seller_stock();
```

4. Transactions

> Transactions needed to assure the integrity of the data.

SQL Reference	Transaction Name
Isolation level	Isolation level of the transaction.
`Complete SQL Code`	

Annex A. SQL Code

> The database scripts are included in this annex to the EBD component.
>
> The database creation script and the population script should be presented as separate elements. The creation script includes the code necessary to build (and rebuild) the database. The population script includes an amount of tuples suitable for testing and with plausible values for the fields of the database.
>
> The complete code of each script must be included in the group's git repository and links added here.

A.1. Database schema

> The complete database creation must be included here and also as a script in the repository.

```
CREATE DATABASE liberato_db;


-- DROP TRIGGERS
DROP TRIGGER IF EXISTS trg_update_book_stock ON order_book;
DROP TRIGGER IF EXISTS trg_prevent_negative_stock ON order_book;
DROP TRIGGER IF EXISTS trg_update_order_total ON order_book;
DROP TRIGGER IF EXISTS trg_auto_block_user ON request;
DROP TRIGGER IF EXISTS trg_sync_seller_stock ON book;

-- DROP FUNCTIONS
DROP FUNCTION IF EXISTS update_book_stock_after_order() CASCADE;
DROP FUNCTION IF EXISTS prevent_negative_stock() CASCADE;
DROP FUNCTION IF EXISTS update_order_total() CASCADE;
DROP FUNCTION IF EXISTS auto_block_user() CASCADE;
DROP FUNCTION IF EXISTS sync_seller_stock() CASCADE;

-- DROP INDEXES
DROP INDEX IF EXISTS idx_notification_user;
```

```sql
DROP INDEX IF EXISTS idx_request_user;
DROP INDEX IF EXISTS idx_request_admin;
DROP INDEX IF EXISTS idx_book_category;
DROP INDEX IF EXISTS idx_book_seller;
DROP INDEX IF EXISTS idx_review_book;
DROP INDEX IF EXISTS idx_wishlist_user;
DROP INDEX IF EXISTS idx_cart_user;
DROP INDEX IF EXISTS idx_order_user;
DROP INDEX IF EXISTS idx_order_book_book;
DROP INDEX IF EXISTS idx_seller_book_seller;

-- DROP TABLES (in reverse dependency order)
DROP TABLE IF EXISTS seller_book CASCADE;
DROP TABLE IF EXISTS order_book CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS payment CASCADE;
DROP TABLE IF EXISTS shopping_cart CASCADE;
DROP TABLE IF EXISTS wishlist CASCADE;
DROP TABLE IF EXISTS review CASCADE;
DROP TABLE IF EXISTS book CASCADE;
DROP TABLE IF EXISTS category CASCADE;
DROP TABLE IF EXISTS request CASCADE;
DROP TABLE IF EXISTS notification CASCADE;
DROP TABLE IF EXISTS seller CASCADE;
DROP TABLE IF EXISTS users CASCADE;
DROP TABLE IF EXISTS admin CASCADE;
DROP TABLE IF EXISTS platform_information CASCADE;

-- DROP ENUM TYPES
DROP TYPE IF EXISTS request_type CASCADE;
DROP TYPE IF EXISTS payment_method_enum CASCADE;



--
-- ENUMERATIONS
--

CREATE TYPE request_type AS ENUM ('UnblockAppeal', 'Seller', 'TicketIssue');
CREATE TYPE payment_method_enum AS ENUM ('MBWay', 'Card', 'Paypal');

--
-- TABLES
--

-- Platform Information
CREATE TABLE platform_information (
    name TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

-- Admin
CREATE TABLE admin (
    email TEXT PRIMARY KEY,
    password_hash TEXT NOT NULL,
    name TEXT NOT NULL UNIQUE
);

-- Users
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    profile_img_url TEXT,
    is_blocked BOOLEAN DEFAULT FALSE,
    CHECK (
```

```
        email ~* '^(?:[a-zA-Z0-9!#$%&''*+/=?^_`{|}~-]+(?:\.[a-zA-Z0-9!#$%&''*+/=?^_`{|}~
    )
);



-- Seller (inherits from Users)
CREATE TABLE seller (
    seller_id INTEGER PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE
);

-- Notification
CREATE TABLE notification (
    notification_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    message TEXT NOT NULL,
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    is_read BOOLEAN DEFAULT FALSE NOT NULL
);

-- Request
CREATE TABLE request (
    request_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    type request_type NOT NULL,
    comment TEXT NOT NULL,
    resolved_by TEXT REFERENCES admin(email) ON DELETE SET NULL
);

-- Category
CREATE TABLE category (
    category_name TEXT PRIMARY KEY
);

-- Book
CREATE TABLE book (
    book_id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    author TEXT NOT NULL,
    price NUMERIC CHECK (price >= 0),
    language TEXT,
    synopsis TEXT,
    category_name TEXT REFERENCES category(category_name) ON DELETE SET NULL,
    seller_id INTEGER REFERENCES seller(seller_id) ON DELETE SET NULL,
    available_stock INTEGER DEFAULT 0 CHECK (available_stock >= 0),
    CONSTRAINT unique_book_name_author UNIQUE (name, author)
);

-- Review
CREATE TABLE review (
    review_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    book_id INTEGER REFERENCES book(book_id) ON DELETE CASCADE,
    rating NUMERIC CHECK (rating >= 0 AND rating <= 5),
    date DATE DEFAULT CURRENT_DATE NOT NULL,
    UNIQUE (user_id, book_id)
);

-- Wishlist
CREATE TABLE wishlist (
    wishlist_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    book_id INTEGER REFERENCES book(book_id) ON DELETE CASCADE,
    UNIQUE (user_id, book_id)
);

-- Shopping Cart
```

```sql
CREATE TABLE shopping_cart (
    cart_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    book_id INTEGER REFERENCES book(book_id) ON DELETE CASCADE,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    UNIQUE (user_id, book_id)
);

-- Payment
CREATE TABLE payment (
    payment_id SERIAL PRIMARY KEY,
    payment_method payment_method_enum NOT NULL,
    amount NUMERIC NOT NULL CHECK (amount >= 0)
);

-- Order
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
    payment_id INTEGER REFERENCES payment(payment_id) ON DELETE SET NULL,
    total_price NUMERIC DEFAULT 0 CHECK (total_price >= 0),
    date DATE DEFAULT CURRENT_DATE NOT NULL
);

-- OrderBook (association between Order and Book)
CREATE TABLE order_book (
    order_id INTEGER REFERENCES orders(order_id) ON DELETE CASCADE,
    book_id INTEGER REFERENCES book(book_id) ON DELETE CASCADE,
    unit_price_at_purchase NUMERIC NOT NULL CHECK (unit_price_at_purchase >= 0),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (order_id, book_id)
);

-- SellerBook (represents seller's available stock)
CREATE TABLE seller_book (
    seller_id INTEGER REFERENCES seller(seller_id) ON DELETE CASCADE,
    book_id INTEGER REFERENCES book(book_id) ON DELETE CASCADE,
    available_stock INTEGER DEFAULT 0 CHECK (available_stock >= 0),
    PRIMARY KEY (seller_id, book_id)
);

-- Performance Indices

CREATE INDEX idx_notification_user ON notification(user_id);
CREATE INDEX idx_request_user ON request(user_id);
CREATE INDEX idx_request_admin ON request(resolved_by);
CREATE INDEX idx_book_category ON book(category_name);
CREATE INDEX idx_book_seller ON book(seller_id);
CREATE INDEX idx_review_book ON review(book_id);
CREATE INDEX idx_wishlist_user ON wishlist(user_id);
CREATE INDEX idx_cart_user ON shopping_cart(user_id);
CREATE INDEX idx_order_user ON orders(user_id);
CREATE INDEX idx_order_book_book ON order_book(book_id);
CREATE INDEX idx_seller_book_seller ON seller_book(seller_id);



-- Triggers
--

-- TRIGGER01: Atualiza o stock do livro após uma nova encomenda
CREATE OR REPLACE FUNCTION update_book_stock_after_order()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE book
    SET available_stock = available_stock - NEW.quantity
    WHERE book_id = NEW.book_id;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_book_stock
AFTER INSERT ON order_book
FOR EACH ROW
EXECUTE FUNCTION update_book_stock_after_order();


-- TRIGGER02: Impede que o stock de um livro se torne negativo
CREATE OR REPLACE FUNCTION prevent_negative_stock()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT available_stock FROM book WHERE book_id = NEW.book_id) - NEW.quantity < 0
        RAISE EXCEPTION 'Stock insuficiente para o livro %', NEW.book_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_negative_stock
BEFORE INSERT ON order_book
FOR EACH ROW
EXECUTE FUNCTION prevent_negative_stock();


-- TRIGGER03: Atualiza automaticamente o total da encomenda
CREATE OR REPLACE FUNCTION update_order_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE orders
    SET total_price = total_price + (NEW.unit_price_at_purchase * NEW.quantity)
    WHERE order_id = NEW.order_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_order_total
AFTER INSERT ON order_book
FOR EACH ROW
EXECUTE FUNCTION update_order_total();


-- TRIGGER04: Bloqueia automaticamente um utilizador quando cria um pedido de desbloquei
CREATE OR REPLACE FUNCTION auto_block_user()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.type = 'UnblockAppeal' THEN
        UPDATE users
        SET is_blocked = TRUE
        WHERE user_id = NEW.user_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_block_user
AFTER INSERT ON request
FOR EACH ROW
EXECUTE FUNCTION auto_block_user();


-- TRIGGER05: Sincroniza o stock entre book e seller_book
CREATE OR REPLACE FUNCTION sync_seller_stock()
RETURNS TRIGGER AS $$
BEGIN
```

```
    UPDATE seller_book
    SET available_stock = NEW.available_stock
    WHERE book_id = NEW.book_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_seller_stock
AFTER UPDATE OF available_stock ON book
FOR EACH ROW
EXECUTE FUNCTION sync_seller_stock();
```

A.2. Database population

> A sample of the database population script, the first 10 lines. The full script is available in the repository.

```
-- Platform Information
INSERT INTO platform_information (name, value) VALUES
('site_name', 'Liberato Books'),
('support_email', 'support@liberato.com'),
('about_us', 'Designed to redefine the reading experience, Liberato brings together tech

-- Admins
INSERT INTO admin (email, password_hash, name) VALUES
('admin1@liberato.com', 'hash_admin1', 'Alice Manager'),
('admin2@liberato.com', 'hash_admin2', 'Bob Supervisor');
```

Revision history

Changes made to the first submission:

1. Added snippet with the DROP tables commands
2. Updated UML

GROUP25134, 09/10/2025

- Dinis Afonso Nunes Pinto, up202306480@up.pt (Editor)
- Iara Catarina Sampaio dos Santos Brás, up202208825@up.pt
- João Paulo Silva Santos, up202006525@up.pt
- Ricardo Alexandre Ribeiro Fernandes, up202304126@up.pt

Comments